Report: Identifying Key Entities in Recipe Data – Syntactic Processing

The rise of the modern culinary industry, apps and e-commerce platforms have created a diverse collection of recipes and culinary knowledge, enabling the development of models for information extraction in the culinary domain.

My objective is to leverage the increasing popularity of online cooking platforms and meal-planning apps by enhancing the user experience. I will achieve this by implementing a **custom-named entity recognition (NER) model** to automatically tag ingredients, quantities and recipe names.

This automation will streamline the process of organizing recipes, improve searchability and enable users to easily find recipes based on available ingredients, portion sizes or specific dietary requirements.

This will ultimately **reduce the labor-intensive and inefficient manual tagging process**, providing a more accessible and efficient way for businesses in the food and recipe industry to manage their recipe databases.

1. Importing Libraries

1.1. Libraries

Several Python libraries were imported to support various stages of my NLP workflow. These libraries are the key and form the foundation for building a robust Named Entity Recognition (NER) model.

They are used for tasks such as data ingestion, data manipulation, logging, pre-processing, feature engineering, model training, evaluation and visualization.

1.2. Logging

A helper function was designed for workflow transparency; to help me log messages for debugging, informational, warning, error and critical messages during the runtime of my notebook.

2. Data Ingestion and Preparation

2.1. Reading and Preparing Recipe Data for Analysis

I implemented a structured data ingestion pipeline that **prioritizes error handling and data integrity**. This process focuses on loading the recipe data and validating its structure for further analysis.

2.1.1. Defining a *load_json_dataframe* function

To begin the data preparation process, a custom reusable function, was defined to open, read and convert JSON files into a Pandas DataFrame.

This approach ensures that data loading can be easily repeated or modified without altering the workflow for JSON recipe data.

2.1.2. Executing the *load_json_dataframe* function

The reusable function was executed with a try-except block ensuring robust error-handling and operational stability.

Error Handling: The function was wrapped in an exception handler, this ensures that file I/O errors (such as "File Not Found") or other errors are caught and logged with a warning, rather than crashing the script.

Validation: Once the data was loaded successfully, to gain an initial understanding of our dataset, five random observations were displayed. This helps us verify that the data has been correctly loaded and converted into a Pandas DataFrame for further analysis.

2.1.3. Description of the DataFrame

After an initial inspection of 5 random observations, further examination of the dataset was conducted to understand the structure, content, dimensions, data types, features, observations etc. The first five rows were displayed to examine the content and its features.

Dimensions (285, 2): The data volume was confirmed with the help of the *.shape* attribute, to ensure that we have a sufficient number of observations for training the model.

Summary and Structure: A pandas method was used to provide a summary of the dataset. It confirmed that the primary features were (input and pos) stored as objects (strings), this was the expected format for textual data.

2.1.4. Missing Value Analysis

A missing value analysis was conducted on the entire dataset across all features to assess the data quality.

The analysis revealed that the dataset is completely clean with zero missing values. **No data imputation or record deletion is required**, allowing us to proceed with the next step, that is Data Manipulation.

2.2. Recipe Data Manipulation

Raw textual data is rarely suitable for NLP or Machine Learning related tasks, especially for sequence labelling tasks like Named Entity Recognition (NER).

The Data Manipulation phase performs the steps required to derive meaningful metrics and validate data integrity and consistency.

In this phase the raw recipe text was transformed with the help of tokenization, which enables sequence-based analysis of the inputs to their corresponding POS labels.

This process allows for the removal of inconsistent and irrelevant recipe data, ensuring high-quality dataset for model training.

2.2.1. Tokenization: Tokens of Input and POS

Tokenization is the process of splitting sentences into smaller units (words) called **tokens**. It helps the computers count word frequencies, assign meaning to words, detect patterns in text.

Splitting: The raw recipe text was divided into smaller pieces (words) by leveraging the split method in pandas. This ensures that a phrase like "6 Karela Bitter" is converted into a list ["6", "Karela", "Bitter"].

The features (*input_tokens* and *pos_tokens*) which were engineered are essential for CRF (Conditional Random Field) models, as they require labeled, structured sequential data to learn context.

2.2.2. Data Integrity: Length Validation

NER is a sequence labelling problem; each input token is assigned a corresponding POS tag. A strict **1 to 1 mapping** is a requirement for NER models. In a valid Named Entity Recognition (NER), these lengths must be identical.

Two new features were derived from existing features. Input length and POS length were derived by calculating the number of elements in each tokenized list respectively.

Mismatch Detected: Upon completion of the feature engineering and validation, 5 observations were identified with inconsistent lengths. These observations were flagged and would be addressed to at a later stage.

The Indexes that require Cleaning and Formatting in the Data-Frame: [17, 27, 79, 164, 207]. The total number of mismatches is 5 observations.

2.2.3. Identifying Unique POS Labels

A custom function was implemented to extract all unique Part-of-Speech (POS) labels from the *pos_tokens* feature of the dataset.

Identifying unique POS labels is an essential step towards data understanding and pre-processing in NLP tasks such as Named Entity Recognition (NER). Models like Conditional Random Field rely heavily on structured input features such as POS tags.

Three labels were present; this reduces the complexity of the problem and should theoretically lead to higher model confidence when predicting common sequences.

Example of Common Sequence: Quantity → Unit → Ingredient,
Example: (Quantity → 1), (Unit → Tablespoon) , (Ingredient → Sugar)

2.2.4. Key Insights from Recipe Data after Validation

Following the validation of the recipe dataset, several inconsistencies were identified between the input tokens (raw recipe data) and their corresponding Part-of-Speech (POS) tags (NER tags).

- **Total Mismatches Identified:** 5 observations

- **Affected Indexes:** 17, 27, 79, 164, 207

- **Specific Discrepancy:** In all the 5 observations, the *input_length* was exactly one token greater than the *pos_length*
- **Example of Discrepancy:** At Index 17, the input contains 15 tokens while the POS list only contains 14.

- **Data Integrity/Technical Justification:** In the context of Named Entity Recognition (NER) and Conditional Random Fields (CRF), data integrity relies on a strict one-to-one mapping between the input tokens and POS labels.

- **The "Offset Risk":** If they do not align correctly, the sequence becomes mathematically broken. This is dangerous because it could lead to incorrect learning, sequence corruption and training failure (dimensional mismatch error).

The identified observations violate the sequence labeling tasks requirement of NER (CRF Models). These observations would have to be cleaned or imputed or dropped during the data pre-processing stage to maintain the data integrity and ensure accurate model training.

2.2.5. Data Cleansing: Removal of Invalid Observations [Data Integrity]

The purpose of this step is to remove inconsistencies from our dataset. Observations which had a mismatch between the number of input tokens and their corresponding POS labels.

Such inconsistencies introduce offset risk and violate the sequence labelling tasks requirement of Named Entity Recognition (CRF Models).

- **The Number of Observations [Before Removal] :** 285
- **The Number of Observations [After Removal] :** 280

A Conditional Random Field (CRF) model is a **probabilistic model** used for sequential labelling tasks. It models the dependency between neighboring labels in a sequence.

This cleaning step guarantees that 100% of the training corpus meets our structural requirement. By strictly feeding the model clean, one-to-one mapped sequential data, we ensure that the probabilities are calculated accurately, free from noise of misaligned data points.

2.2.6. Updating Sequence Lengths (Input Length and POS Length)

Two existing features were updated for the purpose of data integrity. Input length and POS length were re-derived by calculating the number of elements in each tokenized list respectively.

Metadata Synchronization ensures that after performing the necessary data transformations and dropping invalid observations, pre-existing metadata can sometimes become unsynchronized. Recalculating these lengths ensures that the dataset's metadata perfectly reflects the true, current state of the tokens.

2.2.7. Final Quality Assurance: Re-validation of Sequence Lengths

This step acts as a final checkpoint. By verifying these lengths, we are absolutely certain that every single observation in our dataset now has a perfect one-to-one mapping.

Automated Quality Assurance (QA) is important to ensure that our dataset is ready for the next major phase of the project. It is a standard practice in data engineering pipelines to verify that data transformation achieved its intended result. This step also prevents silent failures.

This also guarantees sequence alignment, since Named Entity Recognition tasks are a sequence labelling problem (1-to-1 mapping).

3. Train–Validation Split

3.1. Perform Train and Validation Split Ratio

To prepare the data for Conditional Random Field (CRF) model, the primary dataset was partitioned into a training set and validation set. The training set and validation set were then further split into predictors and target variables respectively.

3.1.1. Dataset Splitting (70:30 Ratio)

To ensure proper model training and evaluation, the dataset was split into training (70%) and validation (30%) subsets. Data Integrity was also successfully verified upon completion of the split.

- **Prevention of Data Leakage:** In Machine Learning, evaluating the model on the same data leads to "overfitting" - where the model memorizes the training data and fails to perform on real-world inputs. Isolating 30% of the dataset creates an unbiased and gives us the true performance of the model.
- **Reproducibility (*random_state= 42*):** This ensures that the notebook is reproducible and is a standard industry practice. It guarantees that if another data scientist runs this notebook, the exact same record would be routed.

3.1.2. Extraction of Training and Validation Sets

Extracting the Input and Output variables is crucial and a necessary step for model training. The Pandas Series were converted to the required format (list). A validation check was performed to ensure Data Integrity.

- **input_tokens** represent the input features (X)
- **pos_tokens** represent the target labels (y)

Extracted separately for both training and validation datasets:-

- **Training Set:**
  - **X_train** → List of input tokens [196 Input Tokens]
  - **y_train** → Corresponding POS labels [196 POS Labels]
- **Validation Set:**
  - **X_val** → List of input tokens [84 Input Tokens]
  - **y_val** → Corresponding POS label [84 POS Labels]

Note: 196 Training samples + 84 Validation samples = 280 total samples

3.1.3. Target Label: Identification of Unique Labels in Training Data

I was able to identify three unique labels within our training corpus: Ingredient, Quantity and Unit.

- **State of the Model:** This confirms that the Conditional Random Field (CRF) is solving a strict 3 Class Multi-Class sequence labelling problem.
- **Flagging Corruption & Auditing:** Python sets enforce uniqueness, this step servers as a checkpoint for human error in the labelling process. If a label had been misspelled in the raw data, the set would have returned a greater number of unique items.

4. Exploratory Data Analysis on Training & Validation Data

4.1. Flattening the Sequence Structures

To facilitate effective exploratory data analysis, a reusable function was introduced to handle the dimensionality reduction of the token arrays. In other words, we had to flatten the nested list into a single list.

- **Exploratory Data Analysis (EDA):** It requires a macro-level view of the entire dataset. Flattening the lists temporarily removes the boundaries of hundreds of recipe data and gives us a unified vocabulary corpus.
- **Algorithmic Efficiency:** List comprehensions were used, since it is implemented in C under the hood, making them significantly faster than standard loops.

4.2. Extraction and Validation of Tokens

To prepare the dataset for detailed exploratory analysis, a custom function was defined to extract, flatten, and validate the *input_tokens* and *pos_tokens* from the training dataset.

- **Enforcing 1:1 Mapping:** Sequence labeling tasks require an exact 1:1 alignment between the input tokens and target labels. If a single token is dropped, shifted or misaligned during the process, it could cause downstream problems and could lead to data corruption.
- **Sample Token Display:** Displays the first 10 tokens for the flattened input tokens and corresponding POS labels. This helps in quick manual verification and understanding of token-label alignment.

  - **Total Input tokens in Training Set:** 7,114
  - **Total POS labels in Training Set:** 7,114

4.3. Categorization of Tokens into Labels (Ingredient, Unit, Quantity)

For the purpose of further exploratory analysis, a custom function was implemented to group tokens into their respective Named Entity Recognition (NER) category.

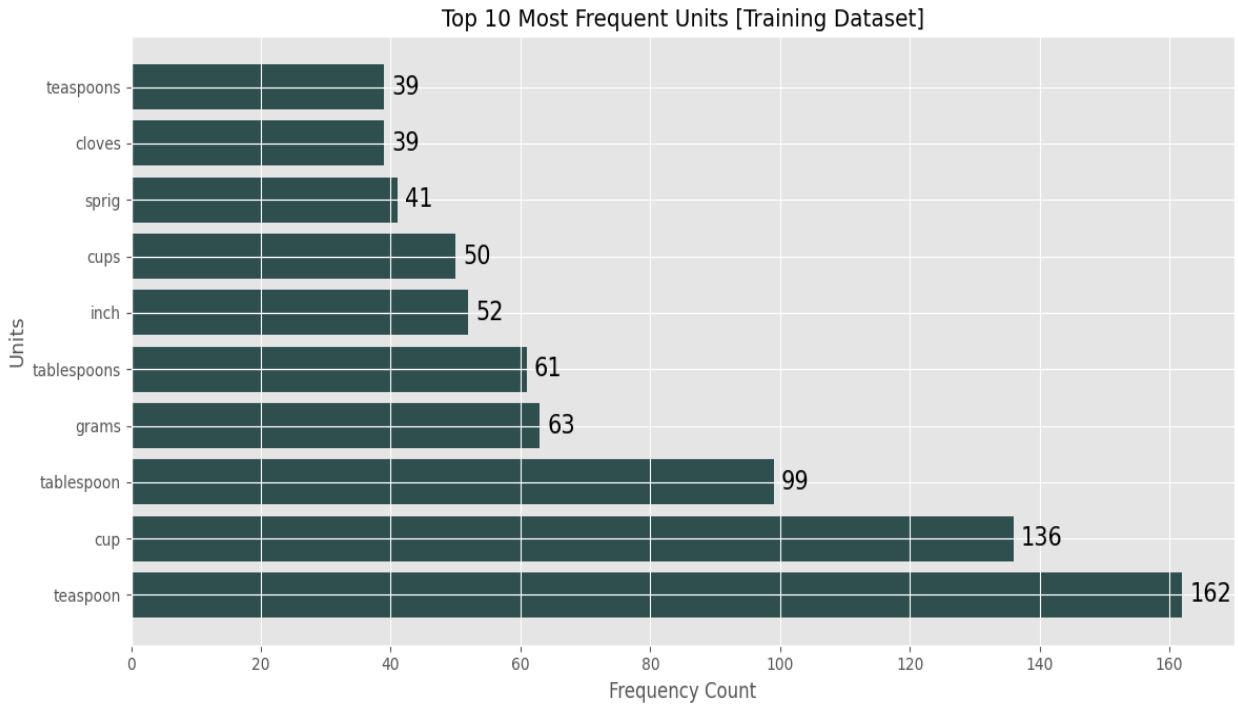
```
First Ten Ingredient Tokens: ['Okra', 'Oil', 'Onion', 'finely', 'chopped', 'Tomato', 'Grated', 'Ginger', 'Garlic', 'Finely']
First Ten Quantity Tokens: ['250', '1', '2', '1/2', '1/4', '200', '2', '1', '1/2', '500']
First Ten Unit Tokens: ['grams', 'teaspoon', 'Teaspoon', 'cup', 'grams', 'tablespoon', 'teaspoon', 'grams', 'teaspoon', 'sprig']
```

- **Data Validation:** The function has a strict validation layer to ensure that the actual labels are a subset of the expected labels. It acts as a "defensive programming" gatekeeper and ensures that the data perfectly meets our project's specific 3-class requirement.
- **Pre-requisite for Targeted EDA:** For an in-depth exploratory analysis about the culinary vocabulary, the data must be isolated. Isolating the data would help us draw insights into the "Top 10 most frequent ingredients" and other exploratory analysis.

4.4. Key Insights: Training Dataset

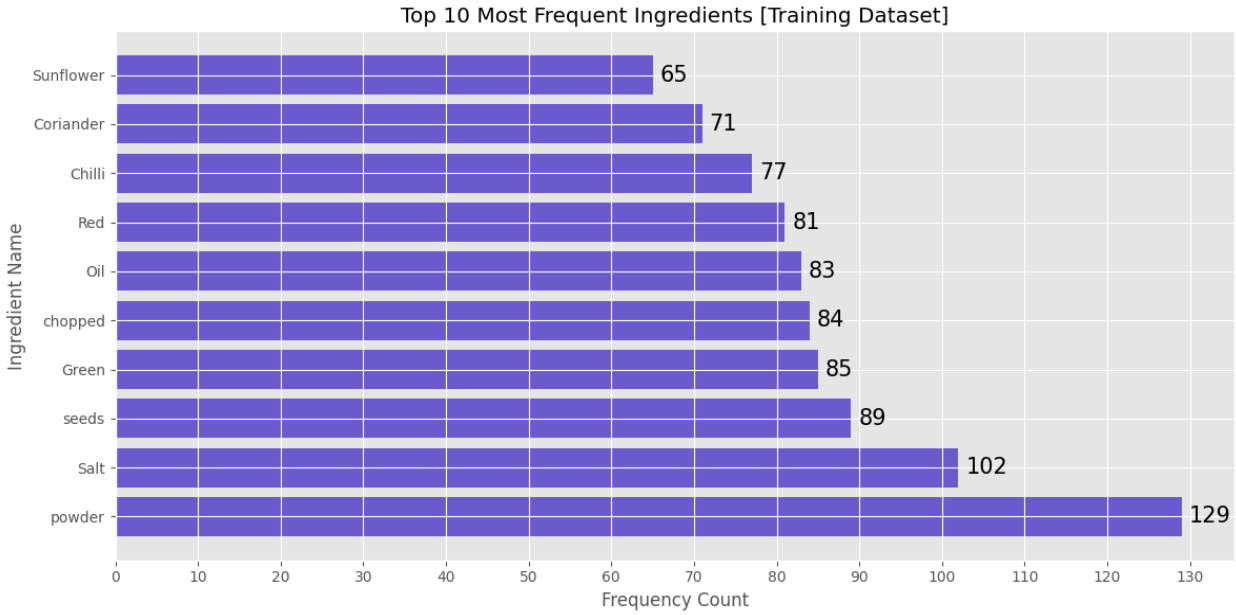
A. Unit Distribution

- **Spoon & Cup Based:** Spoon-based (teaspoon, tablespoon) and Cup-based (cup) measurements dominate the top-ranks.
- **Need for Stemming/Lemmatization:** Tokenization has treated (teaspoon, teaspoons), (cup, cups), (tablespoon, tablespoons) as different words / tokens.



B. Ingredient Distribution

- **Adjectives as Ingredients:** Words like (Green, Red, Chopped) are adjectives used to describe.
- **Multi-Word:** "Red Chilli Powder" or "Chopped Coriander" or "Sunflower Seeds" have been broken into individual tokens.
- **Regional Cuisine:** The Top ingredients suggest an Indian or South Asian influence in the recipe dataset.



C. Quantity Distribution

- **Low-value skew:** The training dataset is heavily skewed towards lower quantities (lower quantities frequently occur). Larger values are present in the bottom ten.
- **Need for Regex or Advanced Library:** Needs robust handling of Quantities (1/2, 1/4) to specifically recognize fractions.

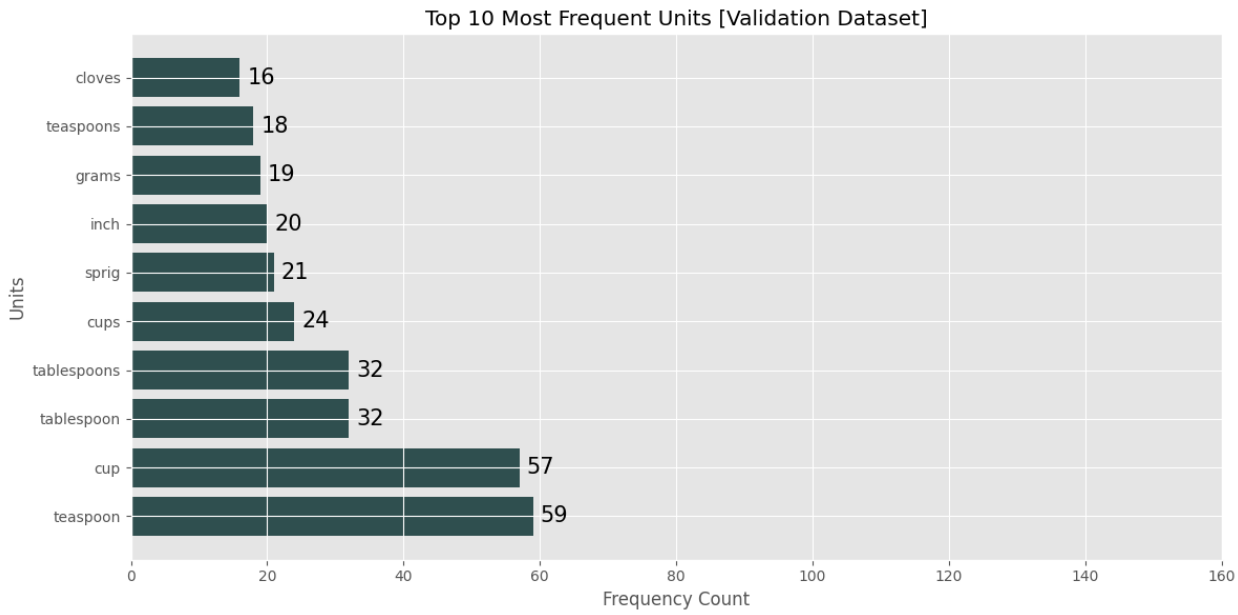
4.5. Key Insights: Validation Dataset

The exploratory data analysis was performed in the same way on the Validation dataset using an identical EDA pipeline.

- **Total Input tokens in Validation Set:** 2,876
- **Total POS labels in Validation Set:** 2,876

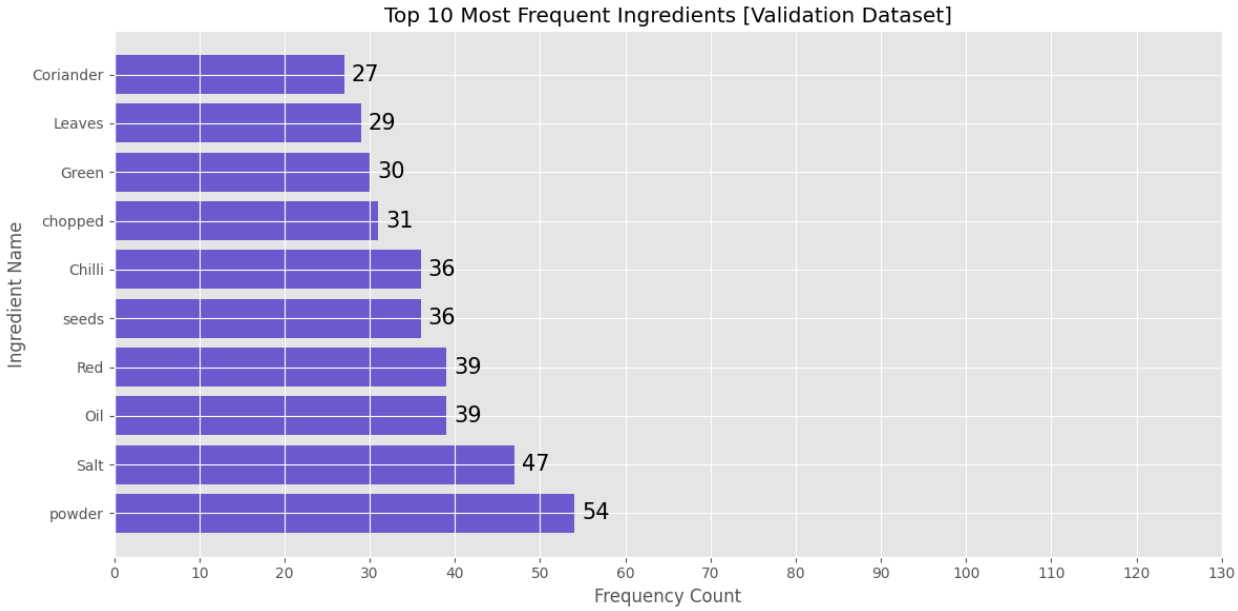
A. Unit Distribution

The Validation data also has the same inconsistencies, confirming that feature engineering is universally required across the entire corpus.
Example: (teaspoon, teaspoons), (tablespoon, tablespoons)



B. Ingredient Distribution

The top 10 ingredients in the validation set are almost identical as the ingredients in the training set. There was a minor introduction of “Leaves” into the validation set. This indicates that the dataset contains multi-word ingredient splits across the entire dataset.



C. Quantity Distribution

The distribution of numerical quantities remains almost identical. Re-asserting the need to handle fractional quantities. Small integer and fractional quantities dominate, matching real-world recipe patterns.

5. Feature Engineering & Extraction For CRF Model

5.1. Developing the word2features Extraction Pipeline

To enable effective extraction of features for our Conditional Random Field (CRFs) model, keyword and pattern-based approaches were implemented to identify unit and quantity tokens in our recipe dataset. This ensures that both structured and unstructured representations of quantities and units in recipe data are addressed to and captured accurately.

5.1.1. Keyword and Pattern Engineering

A. Unit Keyword Design and Engineering

During the Exploratory Data Analysis, we observed that Units appeared in different formats. To capture this without confusing the model, the keywords were divided into two types:

- **Word-form:** They capture the full-standard form of words. (Example: Gram, Centimetre, Tablespoon, Ounce)
- **Short Forms:** They capture the abbreviations of words. (Example: gm, cm, tsp, tbsp, oz)

B. Quantity Keyword Design and Engineering

In many cooking recipes and in real-world data, quantity is usually represented in words rather than digits. Feature sets were engineered to capture this:

- **Word-form:** Non-numeric quantity indicators were defined. It included number (word-form), ordinals, fractional terms, indefinite quantities, article-based quantities and special terms.
- **Key Insight:** During the EDA phase, I was able to identify that indefinite articles act as an integer in some recipe contexts (Example: “Add **a pinch** of salt”). These were added to the keyword set, preventing the CRF model from missing implied singular quantities.

C. Regex for Messy Quantity Tokens in Recipe Data

Real-world recipe quantities are noisy, contain fractions, mixed fractions, positive/negatives, ranges and symbols. A highly efficient and optimized Regex pattern was constructed to handle the variance in recipe data. Instead of writing rigid rules, a flexible regex operation was used to handle all the required edge cases.

1. **Integers:** To Match the Base Number (**Example:** “19”, “200”)
2. **Fractions:** To Match Simple Fractions (**Example:** “1/3”, “2/4”)
3. **Mixed Fractions:** Complex Fractions (**Example:** “1-1/2”)
4. **Ranges:** To Match String as a Range (**Example:** “0-2”, “3-5”)
5. **Decimals:** To Match Floating Point Numbers(**Example:** “2.2”)
6. **Special Symbols:** Handles Special cases (**Example:** “+2/3”)

D. Quality Validation: Regex Patterns

To prove the quantity detection logic actually works as expected, I ran some unit-test which was a success.

I validated the detection logic on the recipe data as well and categorized it into:

- **Keyword-based matches:** Pre-defined string quantities
- **Regex-based matches:** Numeric quantities based on quantity detection logic.
- **Miscellaneous Tokens:** Noise

Key Insight: The combination of Key-word based matching and Regex-based matching provided the necessary coverage needed for quantities. The minor noise highlights the importance of feature engineering.

E. Use of spaCy for Natural Language Processing Tasks

For extraction of deeper meaning from our recipe tokens, the ***en_core_web_sm*** English pipeline from spaCy was integrated.

It supports Token-level processing and Linguistic features such as Part-of-Speech (POS) tags which are used for downstream feature extraction.

These features would later be used by our CRF model to improve its ability to distinguish between features like ingredient, unit and quantity.

5.1.2. Feature Functions defined for Conditional Random Field Model

`*word2features*`: For effective training of a CRF model, a feature extraction pipeline was implemented to generate features at a **token-level** for training the model. The function processes individual tokens within a recipe sentence and generates a feature dictionary for each token which captures the lexical, syntactic, domain-specific, contextual properties.

- **Lexical Properties:** The model captures the token-level attributes. These properties refer to the token as an independent unit without relying on grammar or surrounding text.
- **Syntactic Properties:** Part-of-Speech tags (UPOS & PTB) and dependency relations are defined to identify the token's grammatical role in a sentence.
- **Domain-Specific Semantic Properties:** To capture the culinary-specific role of the token and help distinguish labels based on their semantic meaning.

These features provide the model a strong foundation for differentiating between ingredients, quantities, and units. The CRF model will be able to learn patterns from unstructured recipe text and accurately classify each token in a sentence into labels such as ingredients, quantities and units.

5.2. Preparation of Recipe-Level Features and Target Variables

5.2.1. Sentence-Level Feature Extraction for CRF Model

`*sent2features*`: Serves as a sequence-level wrapper for the extraction of token-level features. Transforms raw input into a structured format for the training of a Conditional Random Field model. This sequential processing ensures that the order of tokens are preserved which is essential for CRF models. To **maximize computational efficiency**, the NLP model is called once for each sentence.

5.2.2. Feature Transformation and Label Extraction

The Training and Validation dataset are prepared for the Conditional Random Field (CRF) algorithm, by transforming them into feature representations and corresponding ground truth labels were converted into structured formats.

- **Feature Transformation:** A list of feature dictionaries was generated for each token for the training and validation sets, while preserving token order. This was done by using the sequence-level ***sent2features*** wrapper.
- **Label Extraction:** The corresponding labels were extracted into list formats to align directly with the feature dictionaries generated in the feature transformation phase.
- **Data Integrity Verification:** The lengths of the features and their corresponding labels were compared and verified to ensure a one-to-one mapping between input features and their corresponding labels. This step ensures that the model is reliable and it prevents training errors.

5.2.3. Target Flattening and Distribution Analysis

For an accurate distribution analysis, the target variable had to be flattened to perform a token-level evaluation. This transformation allows for accurate frequency aggregation of the target classes.

This step is crucial since it provides a quick analysis of class distribution across the dataset, helping us identify class imbalances.

- **Ingredient:** 5,323 tokens
- **Quantity:** 980 tokens
- **Unit:** 811 tokens

5.2.4. Handling Class Imbalance via Inverse Frequency Weighting

Initial Distribution Analysis revealed that **74.82%** of the total tokens in my training dataset was heavily skewed towards the “Ingredient” class, which is common in recipe data. A custom weighting strategy (Inverse Frequency Weighting) was implemented to prevent the Conditional Random Field model from defaulting to the majority class.

Step 1 (Compute Base Weights): By leveraging Inverse Frequency Weighting, Rarer classes received higher weights.
The idea is to force the model to prioritize rarer labels (Quantity & Unit).

- **Rarer Labels (Smaller Count) ----->** Higher Weightage
- **Common Labels (Higher Count) ----->** Lower Weightage

Step 2 (Penalizing Majority Class): Majority Class like “Ingredient” was penalized by a penalty factor of 0.5. This would reduce the influence of the class “Ingredient” during Training.

Step 3 (Feature Injection): A custom function was engineered to map the computed weights into the relevant token feature dictionaries. By injecting ***y_weight_label*** into the training and validation sets, the model is provided with the necessary significance level for each token. This ensures that a majority class doesn't impact the model's predictive performance.

6. Model Building and Training

The Conditional Random Field (CRF) algorithm was developed using custom-weighted feature sets. Optimized hyperparameters and regularization techniques were implemented to improve generalization and prevent overfitting. Overall, the model generalized well and demonstrated excellent learning capability, it achieved a near-perfect performance on the training dataset.

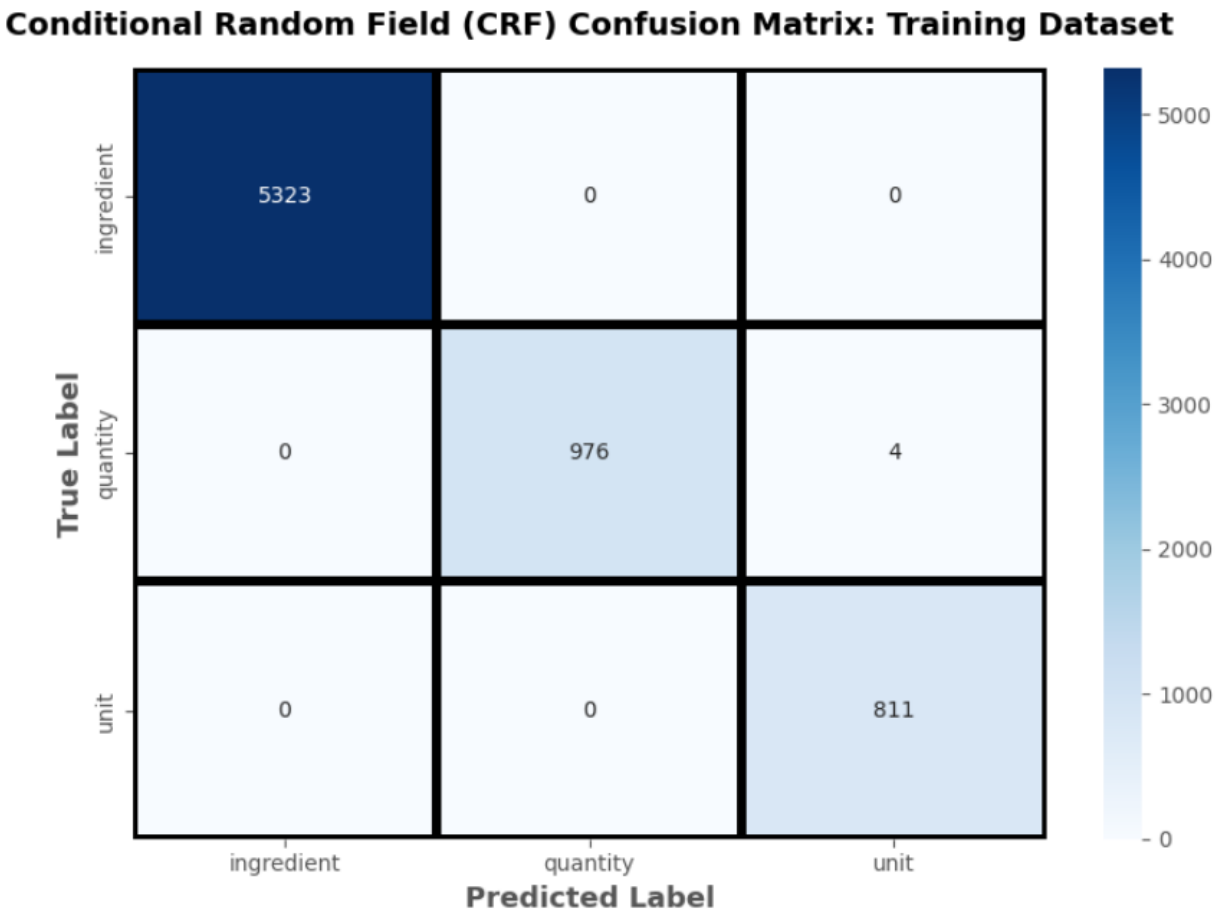
6.1. Initializing and Training of the Conditional Random Field (CRF) Model

The CRF Model was initialized using **L-BFGS optimization algorithm** for memory efficiency, scalability, and second-order convergence. Hyperparameters were carefully tuned to prevent overfitting through regularization techniques like **L1 and L2 regularization**. To learn the best possible weights for my feature set, a **maximum of 100 iterations** was used. The model was successfully trained on the dataset by using the **weighted training features** and their respective labels without any errors. The entire training corpus consisted of 196 Recipe sequences (7,114 Tokens).

6.2. Evaluation of Training Dataset using CRF model

The evaluation of the CRF model on the training dataset, gave me outstanding results. The **Classification Report** displayed **precision, recall and F1-scores of 1.0** across all labels [ingredients, quantity and unit], indicating excellent learning capabilities at a token-level.

Visual analysis of the **Confusion Matrix** confirmed that it's a high-performance model, which could **successfully classify 100% of the ingredient (5,323) and unit (811) tokens**. A negligible margin of error was observed where the model **misclassified 4 out of 980 quantity tokens**. The Confusion Matrix and Classification report on the training dataset indicated that our **model is ready for testing on the validation dataset.**



6.3. Saving the Conditional Random Field Model

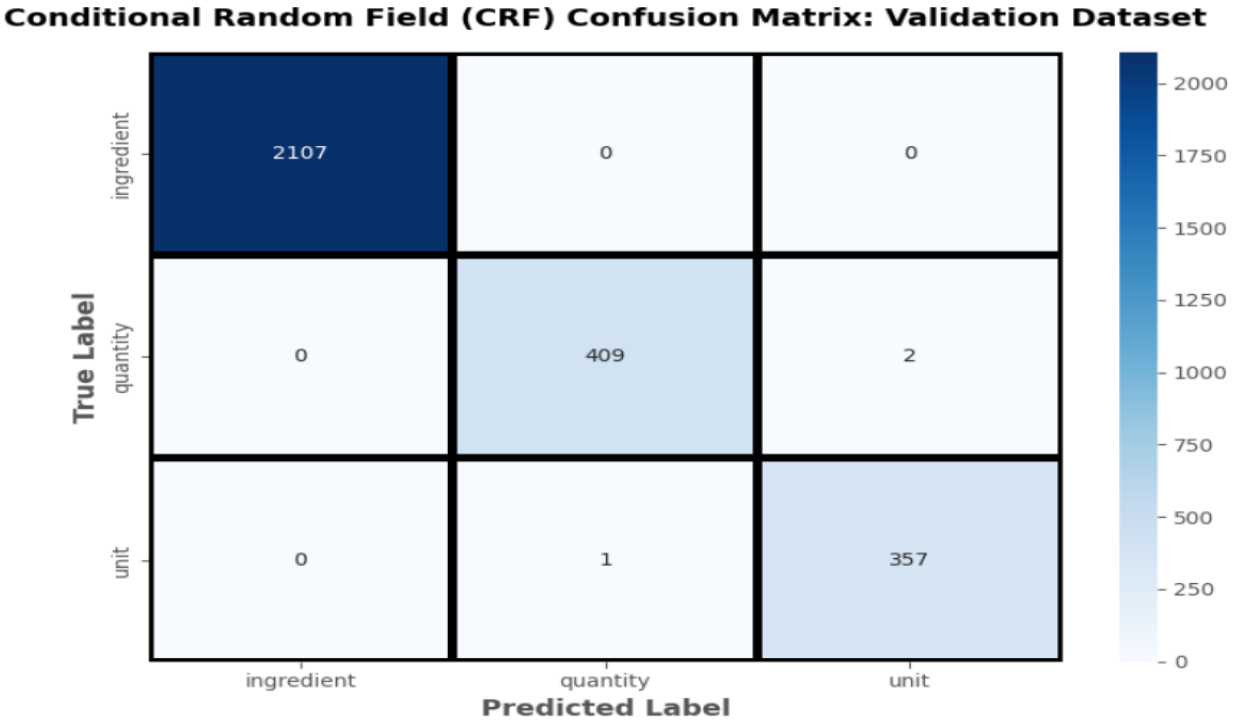
After the CRF model has been trained and evaluated on the training dataset, the next-step involves saving the model, so that it can be re-used without the need for re-training it in the future. The model was successfully exported as a persistent file named ***crf_model.pkl*** using the ***joblib*** library. This step ensures that the model is ready for production deployment and sequence prediction tasks.

7. Prediction and Model Evaluation

My pre-trained CRF Model was evaluated on its performance to predict unseen sequences in the validation dataset. This is done to ensure that the model had actually generalized well and not overfitted on the training dataset. The model once again demonstrated high performance as per the **Classification Report and Confusion Matrix** which was generated. The model's performance was judged on the validation set which comprised of **84 recipe sequences (2,876 tokens)**.

- **Ingredient:** Achieved a **perfect classification rate**, the model was correctly able to identify all 2,107 ingredient tokens.
- **Quantity:** The model **correctly classified 409/411 tokens**, with a minor misclassification of 2 tokens incorrectly predicted as unit.
- **Unit:** The model **correctly classified 357/358 tokens**, with a minor misclassification of 1 token incorrectly predicted as quantity. This resulted in a **slightly lower precision score of 0.99** for the unit class in the classification report which was generated.

Overall, the CRF model committed only **3 misclassifications out of the 2,876 validation tokens**. This near-perfect performance indicates that the inverse frequency weighting strategy engineered during the feature preparation phase was highly effective in handling class imbalance. It allowed the model to isolate and effectively distinguish between the three classes (“Ingredient”, “Quantity”, “Unit”) rather than defaulting to the majority class (“Ingredient”).



8. Error Analysis on Validation Data

To better understand the performance of the Conditional Random Field (CRF) model, an in-depth error analysis was conducted on the validation dataset to examine the misclassified tokens at a micro and macro level. The analysis conducted includes both overall model accuracy and label-specific weaknesses.

8.1. Overall Token-Level Analysis

An error extraction pipeline was implemented to isolate the errors/misclassifications. 3 Misclassifications were recorded out of 2,876 validation tokens. The low error rate indicates the validity of our weighting strategy and confirms that the model is highly accurate even on unseen sequences of recipe data.

- **Tokens Predicted Successfully:** 2,873 / 2,876 Tokens.
- **Token-Level Accuracy:** 99.90%

8.2. Label-Wise (Macro) and Contextual (Micro) Analysis

A label-wise targeted breakdown reveals that **ingredient achieved an accuracy rate of 100%**, while quantity and unit showed 99.51% and 99.72% accuracy rate respectively. The CRF model demonstrated high-level of contextual learning, almost as good as a human, but it encountered minor confusions at the intersection of quantities and units.

- **Unit Accuracy (99.72%)**
  - **1 out of 358-unit tokens was misclassified**, leading to an **accuracy rate of 99.72%.**
  - The model predicted the token **"a"** as a quantity instead of a unit.
  - As per the context, these are not true model errors and is rather a logical linguistic interpretation considering the ambiguity.
  - This suggests that the model may capture semantic relationships more accurately than ground truth labels in some edge cases.

- **Quantity Accuracy (99.51%)**
  - **2 out of 411 quantity tokens were misclassified**, leading to an **accuracy rate of 99.51%**.
  - As per the context the tokens **"pinch" and "clove"** highlight a critical and common challenge of Natural Language Processing for Recipe data. Words can act differently depending on the context and are often used interchangeably.
  - **"Pinch" and "Clove"** can be used as both a unit or a quantity depending on the context.

8.3. Insights on Error Analysis [Validation Dataset]

- **“a” --> defined as a Quantity keyword** [Ground Truth Label – Unit]
  - **Note:** a usually represents quantity, but edge cases do exist.
  - In the grammatical sense it’s an indefinite article, but from a culinary point of view it usually acts as a direct numerical substitution for one.
  - **Example:** “A cup of sugar”, “A pinch of salt”. Therefore classifying it as a quantity keyword was a highly accurate rule for recipe data, even though some edge cases exist where human annotators might label them differently.

- **“pinch” --> defined as a Unit keyword** [Ground Truth Label – Quantity]
  - From a Culinary Point of View, a pinch is usually a unit of measurement.
  - It dictates the exact volume of the ingredient being added.
  - **Example:** “A pinch of salt”

- **“cloves” --> defined as a Unit keyword** [Ground Truth Label – Quantity]
  - It can be an ingredient (dried spice).
  - From a Culinary Point of View, cloves is usually a unit. It dictates the exact volume of the ingredient being added.
  - **Example:** “3 Cloves Garlic”

8.4. Insights from the Validation Dataset

- **Mitigation of Class Imbalance [Inverse Frequency Weighting]:** The validation dataset exhibited a heavy class imbalance which was mitigated by introducing a weighted feature dictionary leveraging Inverse Frequency Weighting. This proves that the technique was able to prevent the CRF algorithm from memorizing or defaulting to the majority-class, allowing it to generalize well on unseen data.
- **Contextual Learning vs Guided Rule-based Learning:** The model is not purely context driven but also guided by rules defined by a human. While this helps improve the accuracy of the model, it could also introduce some bias in cases where words have multiple meanings. Let's take an example of the misclassified tokens “pinch” and “clove” can represent both a “quantity” and “unit” depending on the context.
- **The Double-Edged Sword of Rule-based Feature Engineering:** An error analysis on the validation dataset revealed 3 misclassified tokens (“a”, “pinch”, “cloves”), but these were not random model failures, but it was a direct result of custom feature engineering. The model trusts engineered rules over surrounding context. This highlights that hardcoded rules can often provide strong & relevant signals, it could sometimes overrule semantic context edge cases.

9. Executive Summary and Conclusion of CRF Model Architecture and Performance

The Conditional Random Field Algorithm was engineered and made ready for deployment using an end-to-end Natural Language (NLP) pipeline. The algorithm was designed to extract structured entities like 'ingredients', 'quantities' and 'units') from unstructured recipe text data.

- **Robust Data Cleansing & Ingestion:** 285 Recipe Sequences were rigorously analyzed, my analysis revealed that there were 0 missing values (0.00%) present across the entire dataset. The Tokenization and Validation check revealed that 5 observations had length mismatch between inputs and POS tags. These irrelevant sequences were dropped to ensure 100% data integrity before proceeding to the 70:30 train-validation split.

- **Strict Data Integrity via 1-to-1 Mapping:** The core architectural principle and success of any CRF pipeline relies on precise sequence-level wrapping of features. Before the Training Phase of a CRF model begins, a strict data integrity verification confirmed that 1-to-1 mapping between the input token features and their corresponding ground truth labels. This helps guarantee alignment and prevents training errors.

- **Strategic Mitigation of Class Imbalance:** Exploratory Data Analysis (EDA) revealed a heavy class imbalance. Ingredients compromised almost 75% of the data. This was neutralized by leveraging custom Inverse Frequency Weighting strategy. A penalty factor of 0.5 was used to penalize the majority class. The weights of each label was then injected into the feature dictionaries, this forced the model to learn boundaries of the minority class (quantity, unit) rather than developing a bias and defaulting to the majority class.

- **Domain-Specific Feature Engineering:** The pipeline was enhanced by rule-based human domain knowledge of culinary labels, ensuring that the model performed as well as a human annotator. By integrating custom Regex Patterns (For complex quantity) and culinary-specific keywords alongside spaCy linguistic features, the model received highly accurate human-based rules and signal to guide sequence and token-level parsing.

- **Optimized Model Training & Persistence:** The Conditional Random Field (CRF) model was initialized and optimized with the L-BFGS optimization algorithm for memory efficiency, scalability, and second-order convergence. Hyperparameters were carefully tuned by using appropriate L1/L2 regularization.

The model was able to converge successfully, achieving a perfect 1.00 F1-score on the training data. The entire training corpus consisted of 196 Recipe sequences (7,114 Tokens). The learned weights were serialized into a highly lightweight artifact (*crf_model.pkl at 0.037 MB*).

- **Exceptional Validation Performance & Explainability:** The pre-trained CRF model was evaluated against unseen validation data which comprised of 84 recipe sequences (2,876 tokens). The CRF model achieved a 99.90% token-level accuracy. A granular micro and macros analysis of the errors/misclassifications revealed that they were not algorithmic failures. Instead the model accurately prioritized the human engineered rules over the structural context. (Example: Classifying pinch and cloves strictly as units, because it was defined as a rule)

- **High-Performance Execution:** The entire end-to-end architecture—encompassing JSON ingestion, 1-to-1 mapping, regex validation, complex feature extraction, inverse frequency weighting (for class imbalance), algorithmic training, validation prediction and error reporting—executed flawlessly in just 22 seconds (As per my logs).

- **Strategic Business Value and Operational Impact:** The objective and purpose of this project was to bridge the gap between unstructured culinary text and convert it into actional, structured data. The automation of the identification of ingredients, quantities and units, that my model provides has a significant impact on the scalability for digital food platforms, recipe search engines and meal-planning applications.
    - **Eliminates labour-intensive manual tagging:** The model helps automate the labour-intensive manual tagging, and enables businesses to manage massive unstructured recipe data with ease and near-perfect precision. Furthermore, it enhances the end-user experience by allowing for advanced features like nutritional analysis based on ingredients, precise dietary filtering and intelligent cooking assistants.